CanItShip: Closing the Feedback Loop on Agent Tooling via Trajectory Analysis and Agentic DevX Metrics

Evgenii Kniazev
Databricks
evgenii.kniazev@databricks.com

Arseny Kravchenko
Databricks
arseny.kravchenko@databricks.com

Igor Rekun
Databricks
igor.rekun@databricks.com

Fabian Jakobs
Databricks
fabian.jakobs@databricks.com

Abstract

When AI agents fail to build production applications, should we improve the agent or its environment? Recent work suggests that environment quality (templates, tools, guidance) matters more than model selection [12]. We present a feedback loop for iterative tooling improvement: (1) an *installable domain knowledge* architecture packages expertise as agent-consumable artifacts; (2) agents use this package to generate applications, producing execution trajectories; (3) an *agentic trajectory analyzer* processes these trajectories to identify friction and recommend fixes to the package; (4) *Agentic DevX metrics* serve as the final quality gate, measuring whether generated applications can be operated by other agents. This approach is designed to generalize across domains; we validate it on Databricks data applications across multiple agent backends (Claude Code and Opencode). On 50 applications, we achieve 90% one-shot build success at $2.33/app. The trajectory analyzer identified concrete improvements (batch operations, clearer tool descriptions, missing examples) that we implemented, demonstrating the feedback loop mechanism through case studies. The architecture is designed for automatic optimization: the analyzer's recommendations target tool descriptions, prompts, and examples, artifacts amenable to techniques like GEPA or DSPy-style prompt tuning, potentially closing the loop without human intervention.

CCS Concepts

• **Software and its engineering → Software testing and debugging**; *Software development process management*.

Keywords

CanItShip, agent-agnostic tooling, agentic code generation, trajectory analysis, developer experience metrics, feedback loop

ACM Reference Format:
Evgenii Kniazev, Arseny Kravchenko, Igor Rekun, and Fabian Jakobs. 2026. CanItShip: Closing the Feedback Loop on Agent Tooling via Trajectory Analysis and Agentic DevX Metrics. In *Proceedings of the International Conference on Evaluation and Assessment in Software Engineering (EASE '26), June 9–12, 2026, Glasgow, United Kingdom.* ACM, New York, NY, USA, 5 pages. https://doi.org/XXXXXXX.XXXXXXX

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
EASE '26, Glasgow, United Kingdom
© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 978-1-4503-XXXX-X/26/06
https://doi.org/XXXXXXX.XXXXXXX

1 Introduction

AI agents can generate functional software, but they lack domain-specific knowledge required for production applications. The conventional response is building specialized agents. We take a different approach: improve what agents have access to.

This approach is motivated by recent findings [12]: when comparing agent performance across environments while holding the model constant, environment quality (templates, tools, guidance) had larger effects than model upgrades. An agent with excellent tooling outperforms a better model with poor tooling.

Accepting that tooling matters raises a question: how do we improve it systematically? Manual inspection doesn't scale. End-state metrics (build pass/fail) don't reveal causes. We need a feedback loop that: (1) captures how agents actually use tooling; (2) identifies friction points and root causes; (3) produces actionable recommendations; (4) verifies improvements against agent-centric criteria.

We instantiate each component:

(1) **Installable Domain Knowledge** (Section 2). An architecture for packaging domain expertise as agent-consumable artifacts, exposing tools via CLI commands compatible with emerging skill-based standards.
(2) **Agentic Trajectory Analyzer** (Section 3). A two-phase system: parallel friction extraction with a fast model followed by agentic synthesis with a reasoning model that has source code access.
(3) **CanItShip: Agentic DevX Metrics** (Section 4). The final quality gate: all nine metrics are evaluated by an evaluator agent across three independent trials, measuring whether another agent, not a human, can reliably operate generated applications.

Validation. We validate across Claude Code (primary) and Opencode on Databricks data applications. On 50 applications: 90% build success, 90% database connectivity, $2.33/app average. The trajectory analyzer identified improvements that we implemented and verified through multiple iterations.

2 Installable Domain Knowledge

Developers already use capable agents (Claude Code, Opencode). Rather than asking them to adopt yet another tool, we bring domain knowledge to the agents they already use. A user installs our

package into their existing environment; the agent gains domain expertise without workflow changes.

The package has three components: context layers that inject domain knowledge progressively, tools exposed via CLI, and a state machine that enforces validation before deployment.

Context layers. Agents have limited context windows. Dumping all domain knowledge upfront wastes tokens and can confuse the model. Instead, we inject context progressively; each layer activates when relevant (Table 1).

Table 1: Context layer injection strategy.

Layer	Content	When Injected
Tools	Tool names/descriptions	Always (protocol-level)
Workflow	Patterns, CLI usage	On first discovery call
Target	App vs job constraints	When target type detected
Template	SDK patterns, examples	After scaffolding

CLI tools. We expose domain functionality through CLI commands, a pattern that Cloudflare [6] and Anthropic [3] found effective, reporting that LLMs perform better writing code to call CLI tools than invoking tools directly. Lifecycle commands (`scaffold`, `validate`, `deploy`) and data exploration commands (schema discovery, batch queries) provide the domain-specific capabilities on top of standard workspace tools.

State machine. Applications cannot deploy unless they pass validation after their most recent modification: `Scaffolded` → `Modified` → `Validated(checksum)` → `Deployed`. Any change after validation requires re-validation, preventing untested code deployment, a common failure mode when agents skip validation.

Agent compatibility. To validate agent-agnosticism, we tested the package across Claude Code (automated, primary) and Opencode with open-source models. The key requirement is function calling capability; any agent that can invoke CLI tools works with our package.

3 Agentic Trajectory Analyzer

The analyzer consumes execution trajectories and recommends package improvements. This closes the loop: agents struggle → we see it in trajectories → we fix the tooling → agents struggle less.

End-state metrics (build success, test pass) don't reveal whether failures stem from model limitations, tool problems, missing domain knowledge, or underspecified prompts. Trajectories (the sequence of reasoning, tool calls, and results) show where things went wrong. An agent retrying the same malformed SQL five times reveals a missing example. An agent calling N tools for N tables reveals a missing batch operation.

Two-phase architecture. We employ a map-reduce approach optimized for cost and quality. In the *map phase*, each trajectory is processed independently by a fast model (Claude Haiku, ~ \$0.001 per trajectory), extracting errors, retries, confusion patterns, and inefficient tool usage. This runs in parallel. In the *agentic synthesis phase*, aggregated patterns go to a reasoning model (Claude Opus) with read-only access to template source code, tool definitions, and prior analysis history, generating prioritized recommendations.

Cost model. For N trajectories: map costs $N \times$ ~\$0.001 (fast model), synthesis costs ~\$0.5–3 (reasoning model, bounded at 50 turns). For 50 apps, total analysis cost was under \$15.

Human judgment on small N. On small datasets, we found that the analyzer can overreact, encountering an issue once or twice and recommending building a helper utility without assessing maintenance cost [4]. We recommend human review of recommendations when $N < 50$, treating the analyzer output as a prioritized error analysis rather than an implementation directive.

4 Agentic DevX Metrics

Agentic DevX is the final quality gate. After the package is improved and agents generate applications, we need to verify: can another agent actually operate this output?

Consider an agent that generates a working application. A human developer can run it: they'll figure out missing environment variables, install unlisted dependencies, work around unclear documentation. An agent *can sometimes* do this too, but it's slow and inefficient, spending many turns on trial-and-error that explicit configuration would eliminate.

Agentic evaluation protocol. All nine metrics are evaluated by deploying an evaluator agent (Claude Haiku) that receives a prompt contract (Table 2) and emits a structured `STATUS: PASS|FAIL` verdict, parsed via regex with fallback inference. Each metric runs *three independent trials*; a metric passes if all three trials succeed. Each trial records turns and token usage. A shell-script fallback exists for environments where the agent SDK is unavailable (e.g., root-only clusters).

Table 2: Agentic evaluation prompt contracts.

Metric	Prompt Contract	Trials
L1: Build	Build app for production; run build scripts or import sanity	3
L2: Runtime	Start app, verify healthcheck responds on port	3
L3: Types	Run static/type checks (pyright, mypy, tsc)	3
L4: Tests	Run test suite; FAIL if no tests exist	3
L5: DB Conn.	Connect to the database through the running app; verify query returns rows	3
L6: Data	Verify app returns real backend data, not mocked or hardcoded	3
L7: UI	Load UI on port; verify page renders without errors	3
D8: Runability	Install deps, start server, hit healthcheck endpoint	3
D9: Deploy	Build container, deploy, verify healthcheck returns 2xx	3

The CanItShip framework. The nine metrics span three tiers: Core Functionality (L1–L4), Platform Integration (L5–L7), and Agentic DevX (D8–D9). The harness is platform-independent: prompt contracts reference generic concepts (database connectivity, healthcheck

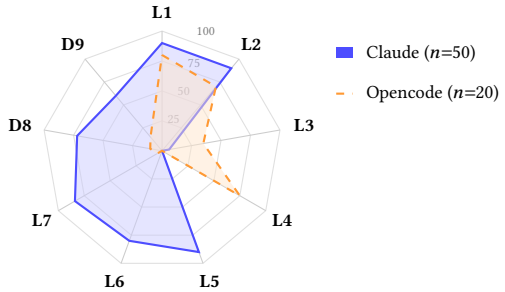

Figure 1: Multi-agent metric profiles (all 9 metrics). Claude Code (solid) achieves broad coverage but dips on L3 (type safety, 6%) and L4 (tests, not yet evaluated). Opencode+Sonnet (dashed, n=20) shows moderate core metrics (L1=80%, L4=75%) and collapses on platform metrics (L5–L7), with D8/D9 provisional.

Table 3: Evaluation results (n=50 Databricks applications).

Tier	Metric	Result
Core	L1: Build Success	45/50 (90%)
	L2: Runtime Success	45/50 (90%)
	L3: Type Safety	3/50 (6%)
	L4: Tests Pass	–*
Platform	L5: DB Connectivity	45/50 (90%)
	L6: Data Returned	40/50 (80%)
	L7: UI Renders	42/50 (84%)
DevX	D8: Runability	36/50 (72%)
	D9: Deployability	30/50 (60%)
Efficiency	Cost per App	$2.33
	Lines of Code	732 avg

*L4 instrumented for Opencode (75%) but not for the primary n=50 Claude evaluation.

endpoints, container deployment) rather than platform-specific APIs.

Generation–evaluation isolation. We observed agents reading evaluation scripts when both resided in the same workspace, producing code that passes checks rather than solving the underlying task. This is a form of reward hacking [2]. We enforce container-level isolation via Dagger [8]: the generation Dockerfile copies only domain package code, explicitly excluding evaluation scripts. Each generation starts from a clean workspace. This mirrors training/test splits in ML; we recommend container-level isolation as a baseline requirement for agent evaluation.

5 Results

We evaluated on 50 Databricks applications spanning dashboards, analytics tools, and business intelligence interfaces. Each was generated from a natural language prompt using Claude Code with CLI-based tool orchestration in containerized execution.

The 90% build/runtime success indicates the core generation pipeline works. The drop from 90% on single-trial metrics to 72%/60% on multi-trial D8/D9 quantifies the gap between "code that works" and "code an agent can reliably operate", the core motivation for

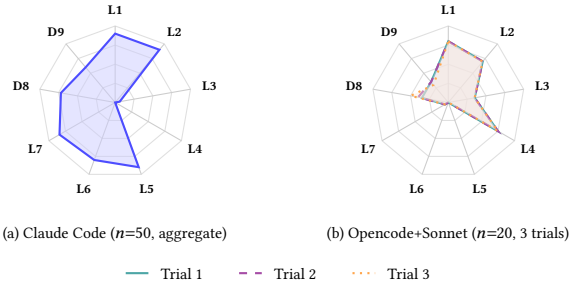

Figure 2: Detailed metric profiles (all 9 metrics). (a) Claude Code: broad coverage; L4 not yet evaluated. (b) Opencode+Sonnet (n=20): three trial-level aggregates show moderate core metrics, platform metrics near 0%, and visible D8/D9 variance across trials.

Agentic DevX. 95% Wilson confidence intervals for key metrics: L1/L2 [79%, 96%], D8 [58%, 83%], D9 [46%, 73%].

Multi-agent comparison. To validate agent-agnosticism, we ran the same CanItShip harness on 20 applications generated by Opencode with Sonnet (Table 4). The pattern is clear: Opencode achieves 80% build and 75% test pass but collapses on platform metrics (L5–L7: 0% DB connectivity, 0% data returned). These applications compile but never connect to Databricks, returning mocked or empty data. This confirms that without installable domain knowledge, even a capable model produces structurally correct but domain-disconnected applications.

Table 4: Multi-agent comparison: Claude Code (n=50) vs Opencode+Sonnet (n=20).

Metric	Claude Code	Opencode	Δ
L1: Build	90%	80%	−10
L2: Runtime	90%	70%	−20
L3: Types	6%	35%	+29
L4: Tests	–*	75%	–
L5: DB Conn.	90%	0%	−90
L6: Data	80%	0%	−80
L7: UI	84%	5%	−79
D8: Runability†	72%	10%	−62
D9: Deploy†	60%	15%	−45

†D8/D9 are provisional harness outputs for the Opencode evaluation.

Agentic metric variance. The 3-trial protocol reveals significant non-determinism in agentic evaluation. Figure 2 shows both agents' metric profiles: Claude Code (a) achieves broad, stable coverage; Opencode (b) shows three independent trials with visible variance on D8 and D9. D8 trial-level pass rates range from 35% to 50% across trials, while D9 ranges from 30% to 35%. Per-app agreement: 50% for D8 (10/20 apps consistent across all trials), 60% for D9 (12/20 consistent). Mean trial-level D8: 41.7% (σ=7.6%); mean trial-level D9: 33.3% (σ=2.9%). Under the strict 3/3 pass criterion, only 2/20 apps pass D8 and 3/20 pass D9, demonstrating why multiple trials are essential for agentic evaluation.

Feedback loop mechanism: case studies. The trajectory analyzer identified concrete friction points that we fixed in the domain

package (Table 5). Agents calling schema discovery once per table (N calls for N tables) revealed a missing batch operation; we added it and observed agents using the batch call in subsequent generations. Repeated SQL syntax errors on `QUALIFY` and `PIVOT` clauses indicated missing dialect-specific examples; we added them to the guidance layer. A tool name mismatch caused agents to hallucinate tool calls; renaming for clarity eliminated the pattern. We demonstrate the mechanism produces actionable recommendations; quantifying cumulative improvement across iterations requires a longitudinal study. These fixes target tool descriptions, prompts, and examples—artifacts amenable to automatic optimization [1, 11]. The full trajectory analysis reports are available in the project repository.

Table 5: Trajectory-identified improvements (implemented).

Pattern Found	Root Cause	Fix Applied
N calls for N tables	Missing batch op	Added batch schema discovery
Repeated SQL errors	Missing examples	Added QUALIFY, PIVOT to guidance
Confusing tool name	Name mismatch	Renamed tool for clarity
Retries on errors	Unclear messages	Added contextual error params
6% type safety	Missing null examples	Added null handling to templates

6 Related Work

Environment over model. Recent work [12] found that environment quality had larger effects than model upgrades when comparing agent performance. SWE-agent [18] similarly demonstrated that agent-computer interfaces significantly affect performance. This motivates our focus on improving tooling rather than agents.

Evaluation gap. Existing benchmarks evaluate code correctness (HumanEval [5], SWE-bench [10]), task completion (WebArena [20], GAIA [15]), or SQL quality (BIRD [13], Spider [19]). None ask whether generated code can be operated by other agents, a critical question for compound AI systems.

Trajectory analysis. Analyzing agent execution traces for debugging is established practice (SWE-agent [18], OpenHands [17]). Our contribution is making the synthesis phase itself agentic: an agent that explores source code to generate recommendations, rather than static aggregation.

Agent evaluation harnesses. Inspect AI [16] and DeepEval [7] provide evaluation infrastructure but focus on correctness rather than deployment readiness. AgentBench [14] spans multiple environments but does not evaluate autonomous deployability. DORA metrics [9] provide the industry standard for delivery performance; to our knowledge, no existing framework maps AI code generation evaluation to DORA.

7 Discussion

Agent-agnostic vs agent-specific. Our approach invests in tooling rather than agents: agents improve rapidly (model upgrades are free to adopters), while domain knowledge is slow to encode and hard to transfer. The tradeoff is control; agent-specific approaches can fine-tune behavior precisely, while we rely on the agent's ability to follow tool descriptions. In practice, this suffices for structured tasks but limits open-ended architectural reasoning.

Toward automatic optimization. Our loop is currently semi-automatic: the analyzer outputs recommendations, a human reviews them. Since recommendations target tool descriptions and prompts, techniques like GEPA [1] and DSPy [11] could potentially close the loop without human intervention.

Limitations. *External validity:* We validate on Databricks; the harness is platform-independent but domain packages require platform-specific implementation. *Internal validity:* n=50 provides initial validation; scaling to 100+ across difficulty tiers is planned. *Construct validity:* All metrics rely on evaluator agent judgment, introducing variance (Figure 2). On small N, the trajectory analyzer can overreact to isolated failures; human review remains essential. The feedback loop is demonstrated qualitatively through case studies (Table 5); longitudinal quantification across multiple iterations with n=100+ is planned.

8 Conclusion

We presented a feedback loop for iterative improvement of agent tooling: installable domain knowledge, an agentic trajectory analyzer, and Agentic DevX metrics. On 50 Databricks applications across multiple agent backends, we achieve 90% build success at $2.33/app. The trajectory analyzer identified concrete improvements that we implemented and verified. When agents fail, improve their environment rather than the agents themselves. The evaluation harness, trajectory analyzer, 50-prompt benchmark, and anonymized artifacts are released under the MIT license.

Acknowledgments

We thank the Databricks Apps team for infrastructure support and the internal hackathon participants whose prompts formed the basis of our benchmark.

References

[1] Lakshya A Agrawal, Shangyin Tan, Dilara Soylu, Noah Ziems, Rishi Khare, Krista Opsahl-Ong, Arnav Singhvi, Herumb Shandilya, Michael J Ryan, Meng Jiang, Christopher Potts, Koushik Sen, Alexandros G Dimakis, Ion Stoica, Dan Klein, Matei Zaharia, and Omar Khattab. 2025. GEPA: Reflective Prompt Evolution Can Outperform Reinforcement Learning. *arXiv preprint arXiv:2507.19457* (2025).

[2] Dario Amodei, Chris Olah, Jacob Steinhardt, Paul Christiano, John Schulman, and Dan Mané. 2016. Concrete Problems in AI Safety. *arXiv preprint arXiv:1606.06565* (2016).

[3] Anthropic. 2024. Tool Use with Claude. https://docs.anthropic.com/en/docs/build-with-claude/tool-use.

[4] Valerii Babushkin and Arseny Kravchenko. 2025. *Machine Learning System Design: With End-to-End Examples.* Manning Publications.

[5] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. 2021. Evaluating Large Language Models Trained on Code. *arXiv preprint arXiv:2107.03374* (2021).

[6] Cloudflare. 2025. Code Mode: Give Agents an Entire API in 1,000 Tokens. https://blog.cloudflare.com/code-mode-mcp/.

[7] Confident AI. 2024. DeepEval: The LLM Evaluation Framework. https://github.com/confident-ai/deepeval.

[8] Dagger Inc. 2024. Dagger: A Portable Devkit for CI/CD Pipelines. https://dagger.io.

[9] Nicole Forsgren, Jez Humble, and Gene Kim. 2018. *Accelerate: The Science of Lean Software and DevOps: Building and Scaling High Performing Technology Organizations.* IT Revolution Press.

[10] Carlos E Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. 2024. SWE-bench: Can Language Models Resolve Real-World GitHub Issues? *arXiv preprint arXiv:2310.06770* (2024).

[11] Omar Khattab, Arnav Singhvi, Paridhi Maheshwari, Zhiyuan Zhang, Keshav Santhanam, Sri Vardhamanan, Saiful Haq, Ashutosh Sharma, Thomas T Joshi, Hanna Mober, et al. 2023. DSPy: Compiling Declarative Language Model Calls into Self-Improving Pipelines. *arXiv preprint arXiv:2310.03714* (2023).

[12] Evgenii Kniazev, Arseny Kravchenko, Igor Rekun, James Broadhead, Nikita Shamgunov, Pranav Sah, Pratik Nichite, and Ivan Yamshchikov. 2025. app.build: A Production Framework for Scaling Agentic Prompt-to-App Generation with Environment Scaffolding. arXiv:2509.03310 [cs.AI]

[13] Jinyang Li, Binyuan Hui, Ge Qu, Jiaxi Yang, Binhua Li, Bowen Li, Bailin Wang, Bowen Qin, Ruiying Geng, Nan Huo, et al. 2023. Can LLM Already Serve as A Database Interface? A BIg Bench for Large-Scale Database Grounded Text-to-SQL. *Advances in Neural Information Processing Systems* 36 (2023).

[14] Xiao Liu, Hao Yu, Hanchen Zhang, Yifan Xu, Xuanyu Lei, Hanyu Lai, Yu Gu, Hangliang Ding, Kaiwen Men, Kejuan Yang, et al. 2023. AgentBench: Evaluating LLMs as Agents. *arXiv preprint arXiv:2308.03688* (2023).

[15] Grégoire Mialon, Roberto Dessì, Maria Lomeli, Christoforos Nalmpantis, Ram Pasunuru, Roberta Raez, Baptiste Rozière, Timo Schick, Jane Dwivedi-Yu, Asli Celikyilmaz, et al. 2023. GAIA: A Benchmark for General AI Assistants. *arXiv preprint arXiv:2311.12983* (2023).

[16] UK AI Safety Institute. 2024. Inspect: A Framework for Large Language Model Evaluations. https://inspect.aisi.org.uk/.

[17] Xingyao Wang, Boxuan Tian, Jiahui Chen, Bowen Li, Ziyi Peng, Tianjun Shi, Robert Hurst, Zijian Yi, Sanjay Jain, Bhavya Vontimitta, et al. 2024. OpenHands: An Open Platform for AI Software Developers as Generalist Agents. *arXiv preprint arXiv:2407.16741* (2024).

[18] John Yang, Carlos E Jimenez, Alexander Wettig, Kilian Liber, Shunyu Yao, Karthik Narasimhan, and Ofir Press. 2024. SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering. *arXiv preprint arXiv:2405.15793* (2024).

[19] Tao Yu, Rui Zhang, Kai Yang, Michihiro Yasunaga, Dongxu Wang, Zifan Li, James Ma, Irene Li, Qingning Yao, Shanelle Roman, et al. 2018. Spider: A Large-Scale Human-Labeled Dataset for Complex and Cross-Domain Semantic Parsing and Text-to-SQL Task. *arXiv preprint arXiv:1809.08887* (2018).

[20] Shuyan Zhou, Frank F Xu, Hao Zhu, Xuhui Zhou, Robert Lo, Abishek Sridhar, Xianyi Cheng, Yonatan Bisk, Daniel Fried, Uri Alon, et al. 2024. WebArena: A Realistic Web Environment for Building Autonomous Agents. *arXiv preprint arXiv:2307.13854* (2024).